

Benchmark de Algoritmos de Ordenação

Édipo Elder Fernandes de Melo

04/10/2026

1 Introdução

Este relatório apresenta uma análise comparativa de cinco algoritmos de ordenação aplicados a uma estrutura de dados em C++ (`std::vector<int>`). O objetivo é medir, de forma padronizada, o tempo de execução médio (em milissegundos) de cada algoritmo em função do tamanho do vetor, considerando três cenários distintos: melhor caso, pior caso e caso médio.

O melhor caso é quando o vetor já está ordenado. O pior caso é quando o vetor está ordenado de forma decrescente. Para testar o caso médio, optamos em criar um vetor com valores aleatórios. Neste último caso de cenário, é possível que o vetor tenha valores repetidos.

1.1 Os algoritmos avaliados

A seguir, resumiremos as características de cada algoritmo testado:

Insertion Sort. Percorre o vetor da esquerda para a direita, inserindo cada elemento na posição correta dentro da parte já ordenada. É eficiente para vetores pequenos ou quase ordenados, mas quadrático no pior caso.

Selection Sort. A cada iteração, seleciona o menor elemento da parte não ordenada e o coloca na posição correta. Realiza sempre o mesmo número de comparações, independentemente da ordem inicial do vetor.

Bubble Sort. Compara pares adjacentes e troca-os se estiverem fora de ordem, repetindo o processo até que nenhuma troca seja necessária. Possui uma otimização que interrompe a execução precocemente se o vetor já estiver ordenado.

Quick Sort. Escolhe um pivô (sempre no meio do range, neste experimento), particiona o vetor em torno dele e ordena recursivamente as subpartes. É um dos algoritmos mais eficientes na prática, com complexidade média $O(n \log n)$.

Merge Sort. Divide o vetor recursivamente pela metade, ordena cada metade e depois as intercala. Garante complexidade $O(n \log n)$ em todos os casos, mas utiliza memória auxiliar.

A tabela a seguir resume a complexidade dos algoritmos no melhor, pior e caso médio:

Tabela 1: Complexidade dos algoritmos de ordenação avaliados

Algoritmo	Melhor	Médio	Pior	Estável?
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Não
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sim
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Não
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Sim

1.2 Cenário do experimento

O programa foi executado num processador **Intel(R) Core(TM) i7-8700** rodando **Linux Pop-OS** com **kernel 7.1.5**, compilado com **g++ 13.3.0** e os seguintes parâmetros:

```
g++ -std=c++23 -march=native -mtune=native -fno-stack-protector benchmark_sort.cpp -o bench_sort
```

O parâmetro **-march=native -mtune=native** diz ao compilador para usar instruções específicas da sua CPU e **-fno-stack-protector** remove canários de pilha (reduz overhead por chamada).

Em todos os testes:

- Foram criados três vetores com tamanho **VECTOR_SIZE** (10000), utilizados de forma padrão nos testes, sendo preenchidos de formas distintas:
 - **Melhor caso:** vetor já ordenado (`melhor_caso[i] = i`).
 - **Pior caso:** vetor em ordem inversa (`pior_caso[VECTOR_SIZE - i - 1] = i`).
 - **Caso médio:** vetor aleatório (`rand() % VECTOR_SIZE`).
- Os tamanhos de vetor testados variam de 500 a 10000, em passos de 500.
- Cada medição foi repetida 10 vezes (**TRIALS**), e calculamos a média dos tempos obtidos.
- Para evitar que o algoritmo receba um vetor já ordenado nos cenários de testes do pior caso e caso médio, a cada nova execução do algoritmo foi criada uma cópia do vetor padrão.

2 A função de benchmark

A fim de evitar repetição de código para cada algoritmo implementado, padronizamos a execução dos testes com um função genérica:

```
typedef void (*algoritmo_ordenacao)(vector<int>& vetor, int end);

inline long benchmark(algoritmo_ordenacao algoritmo, vector<int> &vetor, int end) {
    ...
}
```

A implementação dos algoritmos foi feita:

1. **Com a mesma assinatura de parâmetros.** Todas as funções recebem (**vetor**, **end**) e retornam **void**. O ponteiro de função **algoritmo_ordenacao** permite chamá-las polimorficamente.
2. **Medição isolada.** O cronômetro só envolve a chamada do algoritmo em si. As instruções para criação da cópia do vetor padrão não são computadas no teste.
3. **Cópia do vetor a cada teste.** Isso evita que o algoritmo receba um vetor já ordenado na execução seguinte.

O **main** cria os três vetores de teste e, para cada função registrada na tabela de **funcoes**, computa a média de tempo de 3 repetições para diferentes tamanhos, gerando um arquivo CSV diretamente utilizável pelo R. Optamos por fazer uma média de 3 execuções uma vez que os algoritmos Bubble Sort e Selection sort levam de 25 a quase 200 segundos de execução.

3 Análise por algoritmo

3.1 Insertion Sort

No melhor caso (vetor já ordenado), o algoritmo percorre o vetor uma única vez, resultando em crescimento linear. No pior caso (ordem inversa) e no caso médio, o comportamento é quadrático, com o pior caso apresentando tempos significativamente maiores.

```
void insertion_sort(vector<int>& vetor, int end){
    int key, holePos;
    // percorrer parte não-ordenada (começa em 1)
    for (int i = 1; i < end; i++) {
        key = vetor[i];
        holePos = i - 1;
```

```

// percorrer (reverso) a parte ordenada até o início ou achar a posição
while (holePos >= 0 && vetor[holePos] > key) {
    vetor[holePos + 1] = vetor[holePos];
    holePos = holePos - 1;
}
// inserir item na posição criada
vetor[holePos + 1] = key;
}
}

```

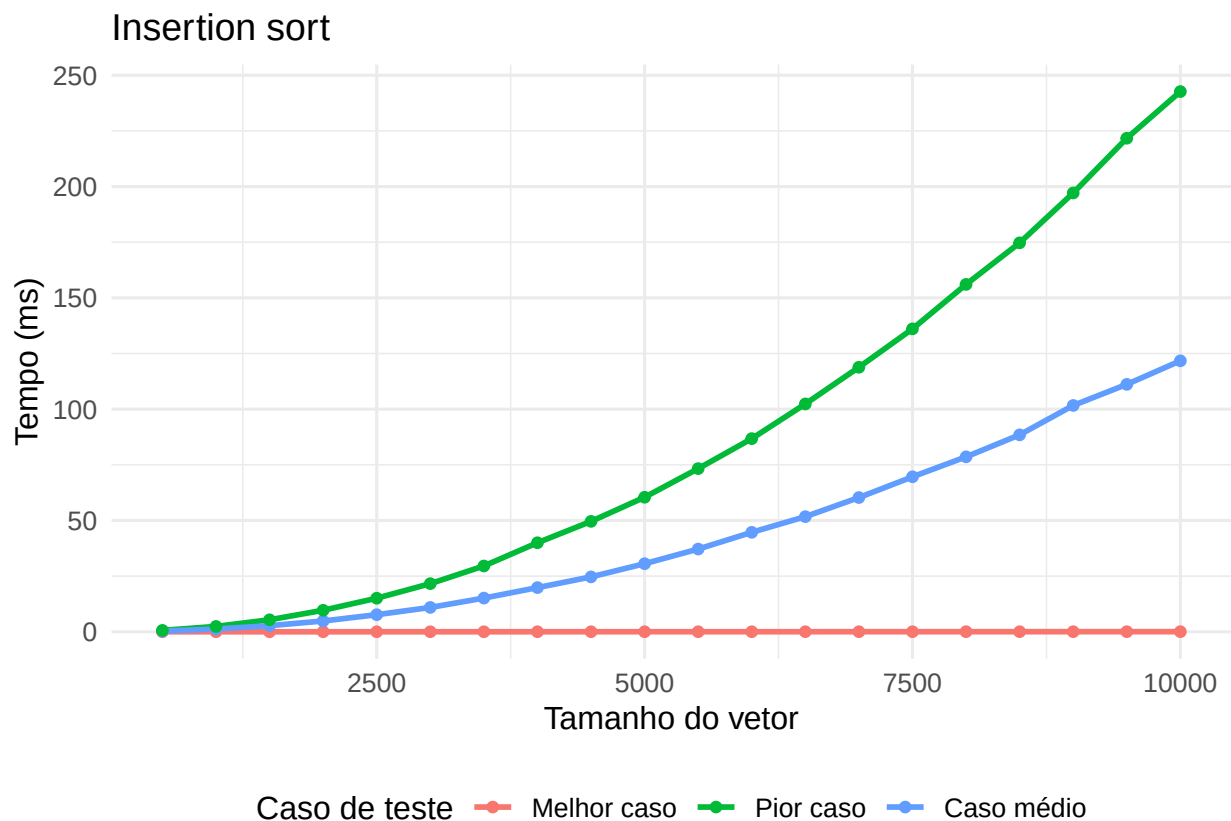


Figura 1: Insertion Sort

3.2 Selection Sort

O Selection Sort realiza sempre o mesmo número de comparações, independentemente da ordem inicial. Contudo, a instrução que determina a troca de valores no vetor (`if (vetor[i] > vetor[j])`) gera um impacto maior quando o vetor contém dados aleatórios, conforme pode ser observado na Figura 2.

```

void selection_sort(vector<int>& vetor, int end){
    for (int i = 0; i < end - 1; i++)
        for (int j = i + 1; j < end; j++)
            if (vetor[i] > vetor[j])
                swap(vetor[i], vetor[j]);
}

```

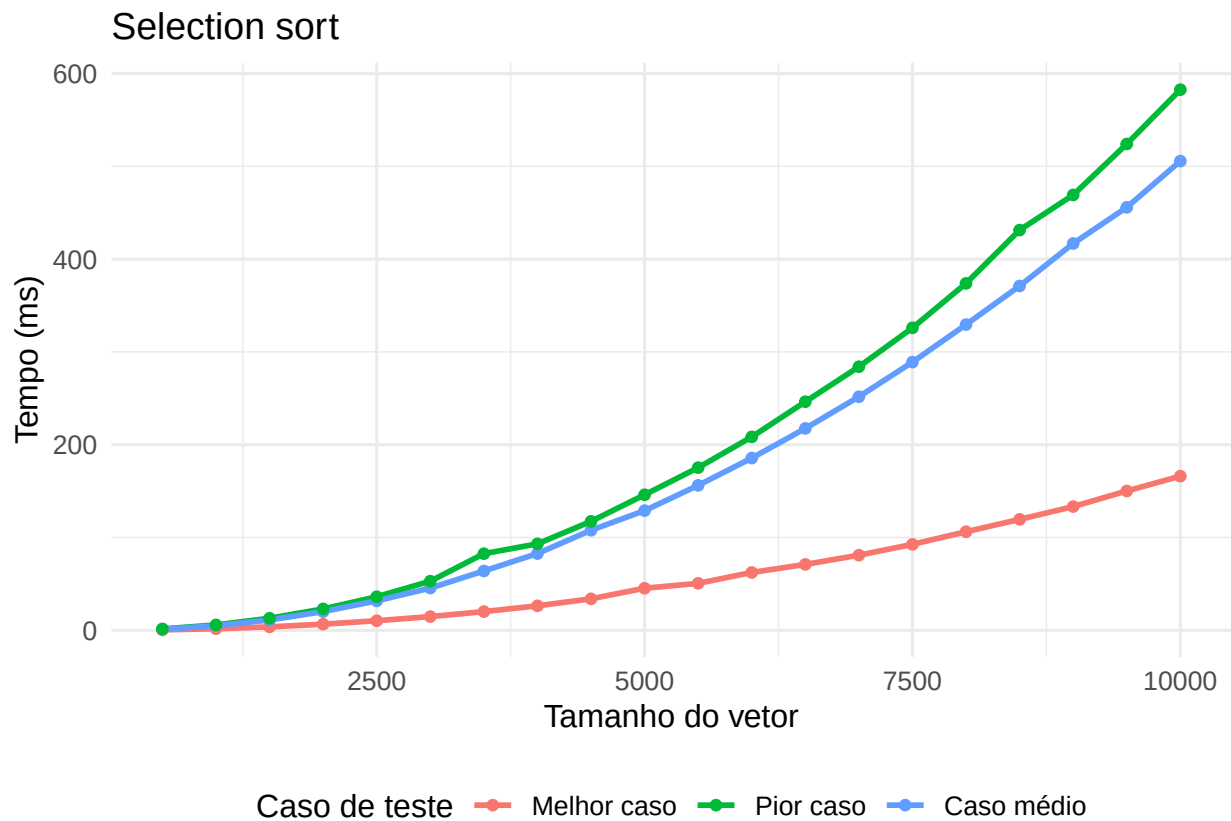


Figura 2: Selection Sort

3.3 Bubble Sort

O Bubble Sort com a otimização de parada precoce apresenta comportamento linear no melhor caso (vetor já ordenado). Nos demais casos, o crescimento é quadrático. AS “quebras” no gráfico do melhor caso podem ser consequência da execução concorrente com outros programas do sistema operacional.

```
void bubble_sort(vector<int>& vetor, int end){
    int j = end - 1;
    bool houveTroca = false;
    do {
        houveTroca = false;
        for (int i = 0; i <= j; i++) {
            if (vetor[i + 1] < vetor[i]) {
                swap(vetor[i], vetor[i + 1]);
                houveTroca = true;
            }
        }
        j = j - 1;
    } while (houveTroca);
}
```

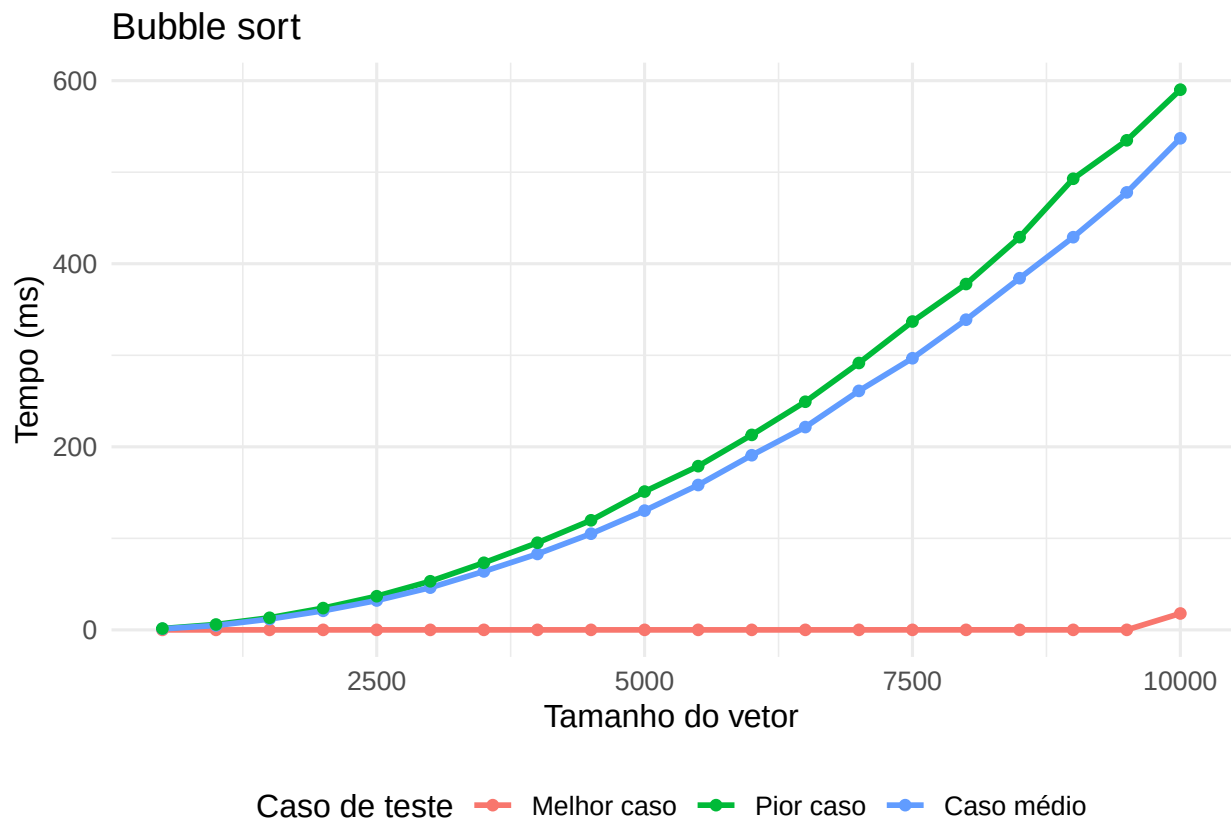


Figura 3: Bubble Sort

3.4 Quick Sort

A escolha do pivô pode interferir no desempenho do algoritmo. Geralmente, é recomendado a escolha de um pivô aleatório. Neste estudo, optamos em escolher o pivô sempre no meio do range para manter uma constância maior entre a execução com vetores de tamanhos diferentes.

O Quick Sort apresenta comportamento $O(n \log n)$ em todos os casos. Observa-se que o pior caso teórico (vetor já ordenado) não se manifesta de forma significativa e a curva fica próxima do melhor caso. Já para o caso aleatório, pela opção de escolha do pivô sempre no meio, a curva foi acima do pior caso. Um fator que pode estar causando esta diferença é a predição do pipeline da CPU (neste caso, um Apple M4).

```
int particao(vector<int>& vetor, int start, int end){
    // int pivo = start + rand() % (end - start + 1); // aleatório entre 0 e end
    //int pivo = end; // pivô sempre no final
    int pivo = start + (end - start) / 2; // pivô sempre no meio
    int x = vetor[pivo];
    swap(vetor[pivo], vetor[end]);
    int i = start - 1;
    for (int j = start; j < end; j++) {
        if (vetor[j] <= x) {
            i++;
            swap(vetor[i], vetor[j]);
        }
    }
    swap(vetor[i + 1], vetor[end]);
    return i + 1;
}

void quick_sort(vector<int>& vetor, int start, int end){
```

```

    if (start < end) {
        int idx = particao(vetor, start, end);
        quick_sort(vetor, start, idx - 1);
        quick_sort(vetor, idx + 1, end);
    }
}

void quick_sort(vector<int>& vetor, int end){
    quick_sort(vetor, 0, end);
}

```

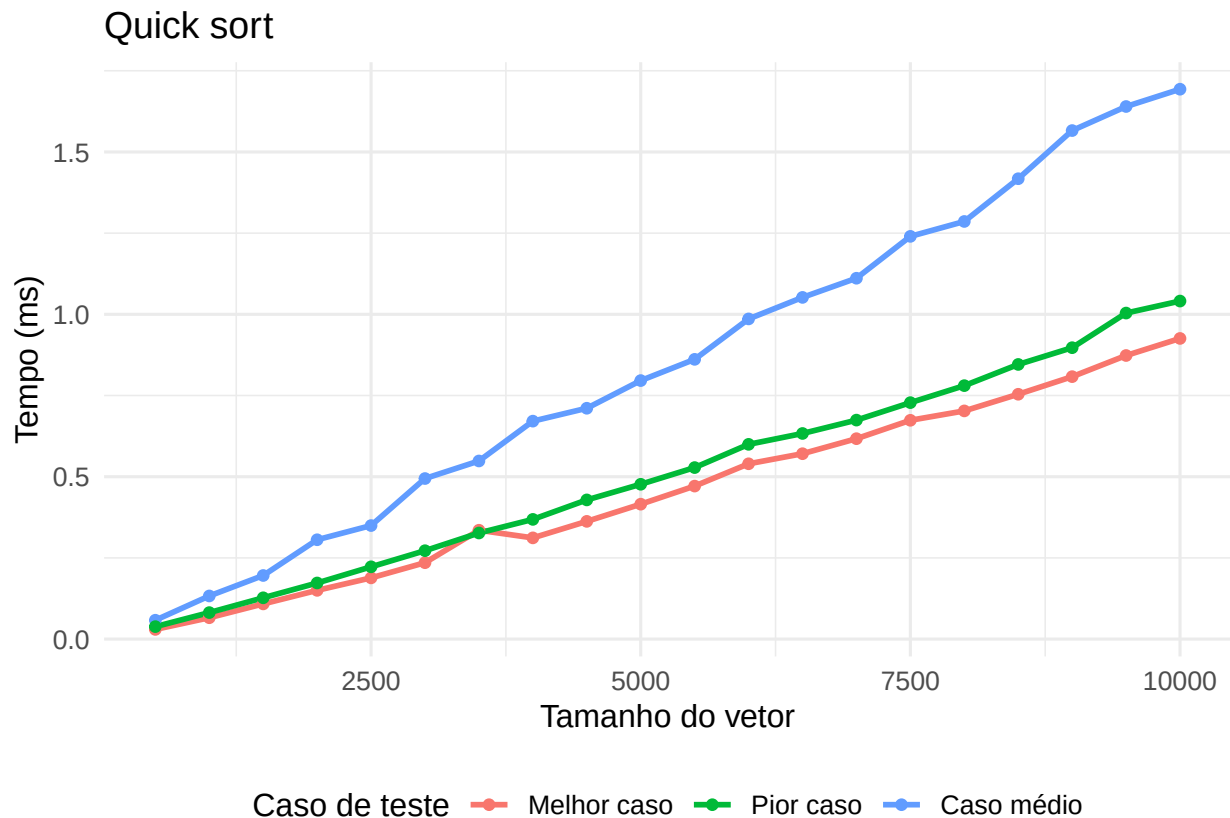


Figura 4: Quick Sort

3.5 Merge Sort

O Merge Sort também apresenta comportamento $O(n \log n)$ em todos os casos, com curvas praticamente coincidentes. Seu tempo é ligeiramente superior ao do Quick Sort devido ao uso de memória auxiliar. De mesma forma, para o caso médio, a curva de tempo de execução é ligeiramente superior.

```

// método para fazer o join dos vetores ordenados
void intercala(vector<int>& left, vector<int>& right, vector<int>& vetor) {
    int i = 0; // índice para left
    int j = 0; // índice para right
    int k = 0; // índice para o vetor resultado
    // Compara elementos e insere o menor em vetor
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) {
            vetor[k++] = left[i++];
        } else {
            vetor[k++] = right[j++];
        }
    }
}

```

```

    }
    // Copia os elementos restantes
    while (i < left.size()) {
        vetor[k++] = left[i++];
    }
    while (j < right.size()) {
        vetor[k++] = right[j++];
    }
}

// rotina recursiva do merge
void merge_sort(vector<int>& vetor, int end) {
    if (end < 2) return;
    int meio = end / 2;
    // vetores esquerda (left) e direita (right)
    vector<int> left(meio);
    vector<int> right(end - meio);
    // Copia os elementos para left e right
    for (int i = 0; i < meio; i++) {
        left[i] = vetor[i];
    }
    for (int i = meio; i < end; i++) {
        right[i - meio] = vetor[i];
    }
    // Chamadas recursivas
    merge_sort(left, meio);
    merge_sort(right, end - meio);
    // Intercala de volta ao vetor
    intercala(left, right, vetor);
}

```

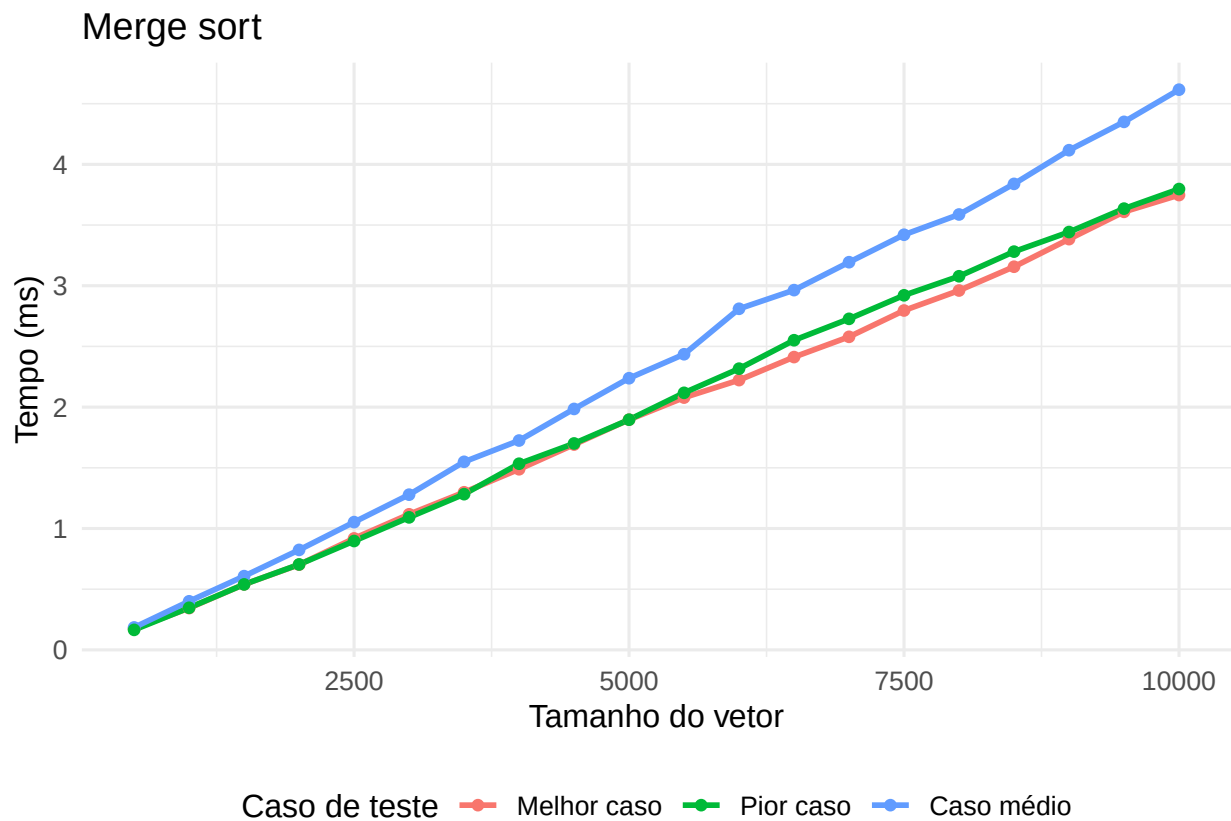


Figura 5: Merge Sort

4 Conclusão

4.1 Melhor caso (vetor já ordenado)

No melhor caso, Insertion Sort e Bubble Sort apresentam crescimento linear, enquanto Selection Sort, Quick Sort e Merge Sort mantêm seus comportamentos característicos. O Insertion Sort é o mais rápido nesse cenário.

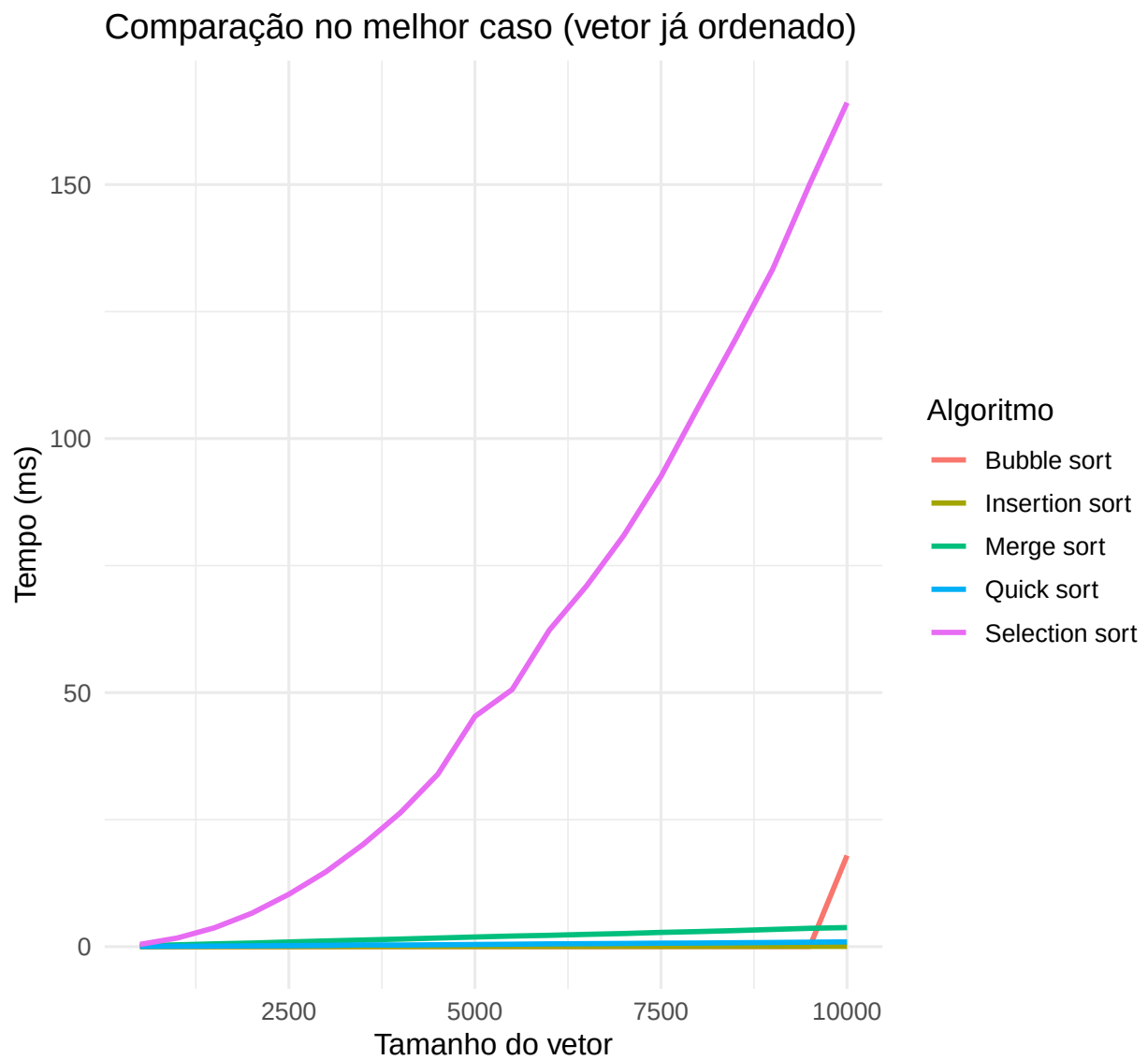


Figura 6: Comparação no melhor caso

4.2 Pior caso (vetor em ordem inversa)

No pior caso, Insertion Sort, Selection Sort e Bubble Sort apresentam crescimento quadrático acentuado, com tempos muito superiores aos do Quick Sort e Merge Sort, que se mantêm em $O(n \log n)$.

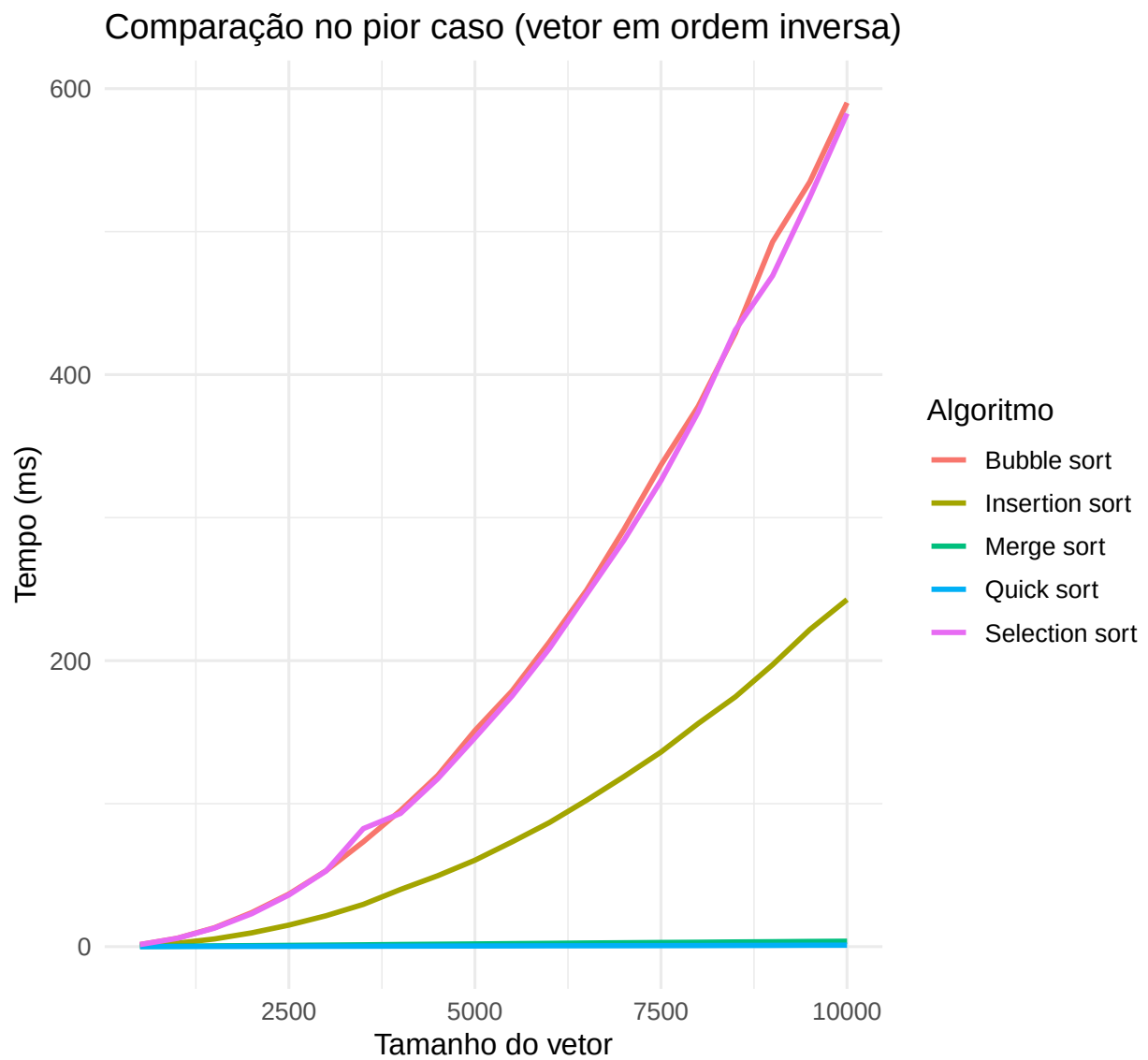


Figura 7: Comparação no pior caso

4.3 Caso médio (vetor aleatório)

No caso médio, o comportamento é semelhante ao pior caso para os algoritmos quadráticos, mas com tempos ligeiramente menores. Quick Sort e Merge Sort continuam sendo as melhores escolhas para vetores grandes.

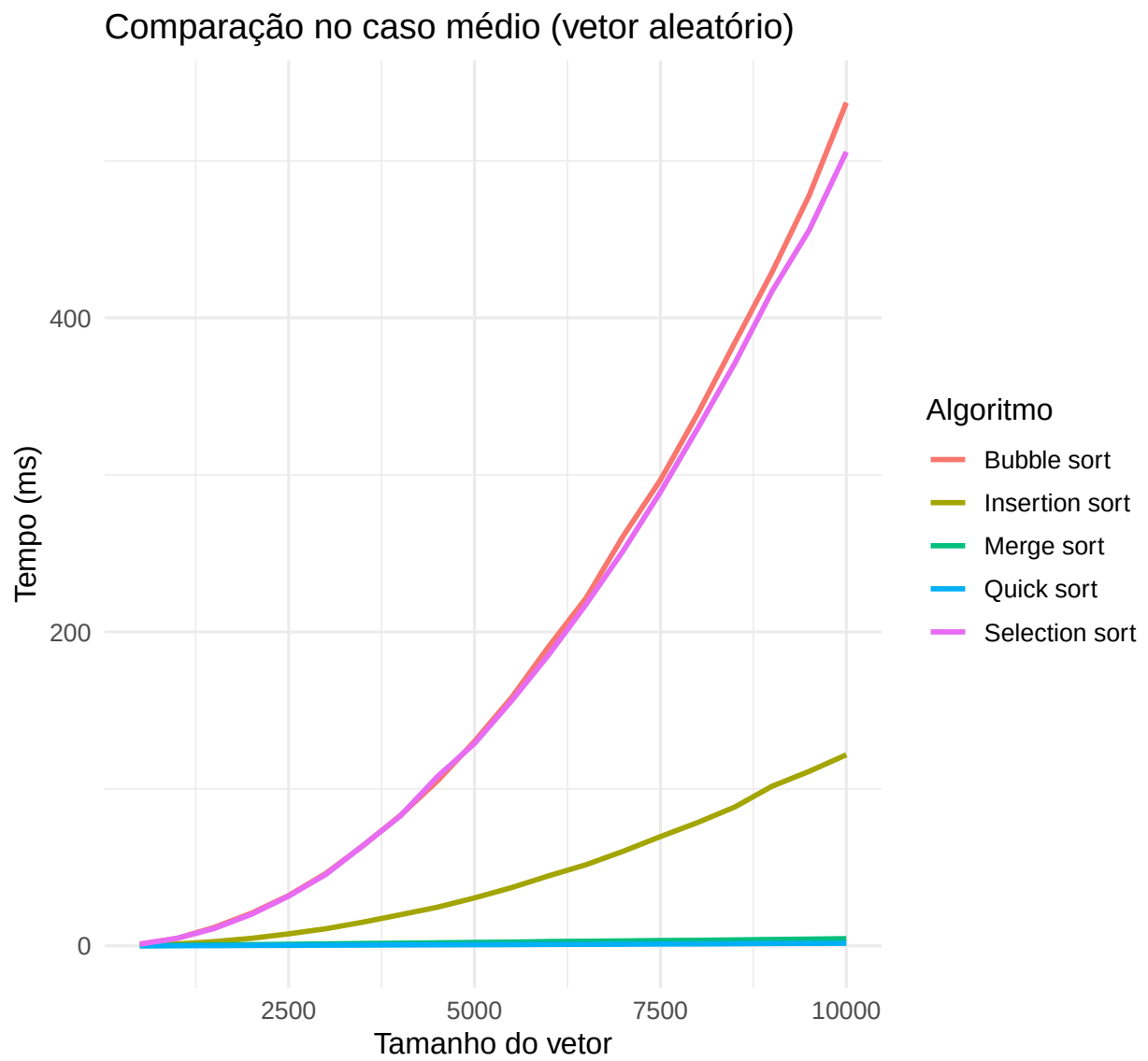


Figura 8: Comparação no caso médio

Algoritmos $O(n \log n)$: Quick Sort e Merge Sort

Melhor, pior e caso médio

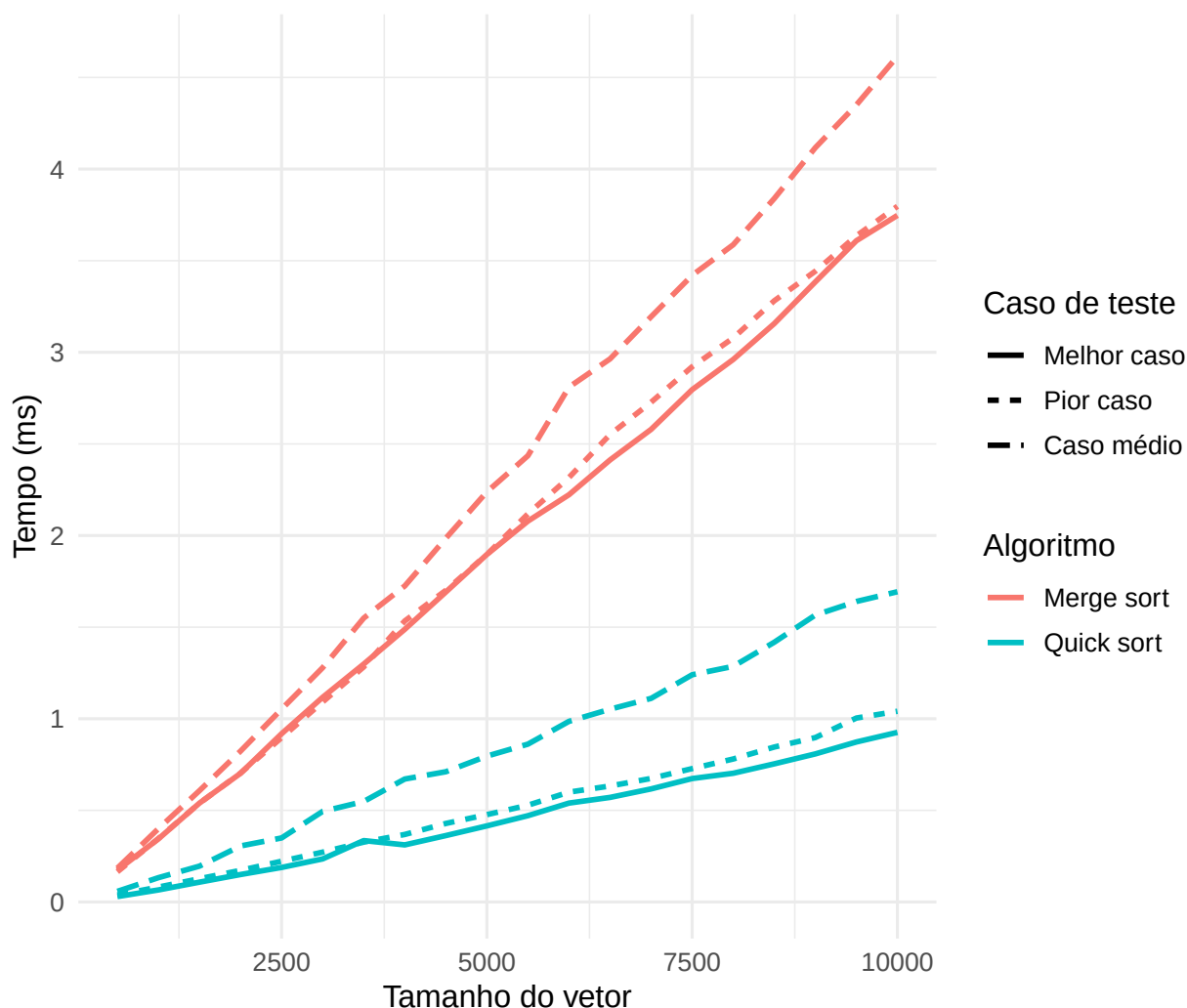


Figura 9: Comparação entre os algoritmos $O(n \log n)$

4.4 Considerações finais

Observamos que, empiricamente, o algoritmo **Quick sort** tem comportamento distinto do comportamento esperado na teoria, com o caso médio ficando acima do pior caso (Figura 4). Supomos que deve-se aos parâmetros utilizados com o compilador (otimização, instruções específicas da CPU, etc.), bem como a predição de instruções do pipeline e cache de instruções pela CPU. CAberia

De modo geral, observamos que, para vetores grandes e desordenados, o **Quick Sort** e o **Merge Sort** são as melhores escolhas, com complexidade $O(n \log n)$ em todos os casos. Na prática, o **Quick sort** é mais rápido que o **Merge sort** (Figura 9), além da vantagem de utilizar menos memória.

O **Insertion Sort** se destaca no melhor caso (vetor já ordenado) com comportamento linear, sendo útil quando os dados estão quase ordenados. Já o **Selection Sort** e o **Bubble Sort**, embora sejam de simples implementação, são ineficientes para vetores grandes, apresentando crescimento quadrático mesmo no melhor caso (no caso do Selection Sort).

Como trabalho futuro, sugerimos um estudo do algoritmo **Quick Sort** com variação do método de escolha do pivô (no fim, no meio, aleatório, etc) bem como quais parâmetros de compilação são os mais indicados para programas que utilizem este algoritmo.

A Tabela de tempos

Tabela 2: Tempos médios de execução (em milissegundos) por algoritmo, caso e tamanho

Tam.	Caso	Insert	Select	Bubble	Quick	Merge
500	Melhor caso	0.002625	0.439524	0.001783	0.029707	0.173712
1000	Melhor caso	0.006529	1.693768	0.003528	0.065697	0.344383
1500	Melhor caso	0.007796	3.704691	0.005304	0.108261	0.539774
2000	Melhor caso	0.010394	6.560671	0.009334	0.150015	0.702258
2500	Melhor caso	0.012991	10.307624	0.008781	0.188418	0.918359
3000	Melhor caso	0.020321	14.755081	0.010654	0.235194	1.117094
3500	Melhor caso	0.018191	20.145422	0.016631	0.335148	1.298443
4000	Melhor caso	0.022175	26.373935	0.019036	0.311609	1.488092
4500	Melhor caso	0.026023	33.912111	0.016476	0.362462	1.691558
5000	Melhor caso	0.025993	45.314830	0.019290	0.415368	1.896551
5500	Melhor caso	0.031144	50.602489	0.022673	0.471106	2.078540
6000	Melhor caso	0.031202	62.313154	0.022993	0.539781	2.222648
6500	Melhor caso	0.033810	71.005517	0.022869	0.570811	2.412312
7000	Melhor caso	0.042185	80.939879	0.024637	0.617046	2.579135
7500	Melhor caso	0.039316	92.590846	0.026399	0.673696	2.796059
8000	Melhor caso	0.043734	106.208716	0.032182	0.702599	2.960698
8500	Melhor caso	0.046987	119.534739	0.031305	0.753899	3.156588
9000	Melhor caso	0.051415	133.316896	0.032159	0.808078	3.383946
9500	Melhor caso	0.051275	150.139257	0.034930	0.873212	3.608870
10000	Melhor caso	0.052322	166.117603	17.926549	0.925722	3.747074
500	Pior caso	0.631482	1.474150	1.473465	0.038295	0.164734
1000	Pior caso	2.401432	5.794490	5.890080	0.081524	0.346900
1500	Pior caso	5.392214	12.967636	13.235980	0.127079	0.539342
2000	Pior caso	9.660840	23.029318	23.804955	0.172719	0.703779
2500	Pior caso	15.084625	36.160768	36.778069	0.222479	0.896848
3000	Pior caso	21.631590	52.939616	53.110005	0.272150	1.091658
3500	Pior caso	29.566700	82.554942	73.313369	0.326617	1.282734
4000	Pior caso	40.000149	93.165044	95.204176	0.368528	1.533739
4500	Pior caso	49.603459	117.413493	119.706445	0.428501	1.700677
5000	Pior caso	60.439801	146.043823	151.111462	0.476711	1.896395
5500	Pior caso	73.296149	175.291043	178.872373	0.528091	2.117794
6000	Pior caso	86.775799	208.438716	213.026397	0.599634	2.316801
6500	Pior caso	102.353435	246.319828	249.252226	0.633090	2.551097
7000	Pior caso	118.831642	284.030883	291.564754	0.674255	2.727290
7500	Pior caso	136.084855	325.957442	336.880543	0.728077	2.921143
8000	Pior caso	156.057176	373.874444	377.844940	0.780398	3.078132
8500	Pior caso	174.730544	431.318043	429.063513	0.845696	3.281056
9000	Pior caso	197.139543	469.191014	492.853770	0.897284	3.442420
9500	Pior caso	221.720502	524.096533	534.840950	1.003921	3.635316
10000	Pior caso	242.679101	582.608529	590.170807	1.040886	3.796776
500	Caso médio	0.300346	1.149615	1.154309	0.058048	0.184829
1000	Caso médio	1.213193	4.840043	4.811186	0.132753	0.400362
1500	Caso médio	2.752136	11.101676	11.791870	0.195749	0.607516
2000	Caso médio	4.848951	20.128758	20.853603	0.305818	0.823576
2500	Caso médio	7.681801	31.719092	32.080626	0.349515	1.052543
3000	Caso médio	10.941828	45.508801	46.163604	0.494355	1.278666
3500	Caso médio	15.143588	63.901572	63.864386	0.548423	1.549332
4000	Caso médio	19.845412	82.701310	82.962464	0.671098	1.724759
4500	Caso médio	24.660679	107.817956	105.060040	0.710769	1.984996
5000	Caso médio	30.588462	128.930775	130.293520	0.795991	2.238083
5500	Caso médio	37.165842	156.191869	158.266074	0.861387	2.435544

6000	Caso médio	44.682141	185.596913	190.812258	0.985863	2.810172
6500	Caso médio	51.727706	217.551871	221.559515	1.052141	2.963892
7000	Caso médio	60.298467	251.692578	261.093301	1.111168	3.193961
7500	Caso médio	69.629089	289.035114	296.769323	1.240100	3.420110
8000	Caso médio	78.585527	329.477474	338.884983	1.285963	3.587025
8500	Caso médio	88.473308	371.064099	384.210043	1.417598	3.839143
9000	Caso médio	101.679857	416.860453	429.019025	1.565930	4.116641
9500	Caso médio	111.149173	455.816418	477.950055	1.639983	4.350041
10000	Caso médio	121.726716	505.665073	537.053839	1.693248	4.615281
